Pixel Avatar DDPM: A Tiny Text-Conditioned Avatar Diffusion from Scratch

Valerio Cassone – ID 601225
Polytechnic University of Bari
Bari, Italy
v.cassone1@studenti.poliba.it

***Abstract*—Recent advancements in text-to-image generative modelling rely heavily on large-scale pretrained backbones, such as CLIP and pretrained variational autoencoders (VAEs). However, these models obscure whether cross-modal alignment originates from the generative diffusion process itself or from inherited language-vision priors. In this work, we present a compact, fully self-contained, pixel-space denoising diffusion probabilistic model (DDPM) that is trained entirely from scratch on the Google Cartoon Set. Our framework operates directly on RGB pixel grids (32×32 and 64×64), and combines a Transformer-based text encoder with four layers and a lightweight U-Net denoiser with spatial cross-attention mechanisms. Neither component uses external pretrained weights. To evaluate systematic concept recombination, we introduce a rigorous compositional out-of-distribution (OOD) benchmark protocol by isolating specific attribute pairs. Quantitative evaluations using Fréchet Inception Distance (FID) and Kernel Inception Distance (KID), alongside hardware profiling, demonstrate a minimal performance gap between in-distribution and out-of-distribution (OOD) test splits. Our findings confirm that lightweight, pixel-space diffusion models trained from scratch can achieve orthogonal concept representations and zero-shot attribute recombination under strict computational constraints. The complete source code, preprocessing scripts, and model configurations are publicly available on GitHub [1].**

I. Introduction

A. Background and Motivation

In recent years, there has been a fundamental shift in text-to-image generative modelling, largely driven by the development and success of Denoising Diffusion Probabilistic Models (DDPMs) [2]. Systems such as Stable Diffusion, Imagen and Midjourney have demonstrated impressive capabilities in synthesising high-fidelity, semantically complex images from natural language descriptions [3]. Contemporary state-of-the-art text-to-image architectures predominantly rely on large, general-purpose, pre-trained backbones, such as CLIP, T5 or pre-trained variational autoencoders (VAEs), to handle multimodal feature extraction and alignment.

While these large-scale pre-trained components produce exceptional visual realism, they essentially function as "black boxes". Consequently, it is challenging to determine whether the observed cross-modal alignment and attribute control originates directly from the generative diffusion process or are merely inherited from the high-capacity representations of pretrained vision-language encoders. Studying compact diffusion architectures that are trained entirely from scratch — without any pretrained backbones — provides complete transparency into the mechanics of cross-attention conditioning, noise estimation, and cross-modal alignment.

B. Research Question

This project investigates the foundational capabilities of lightweight, self-contained diffusion architectures under strict computational limits. Specifically, we address the following primary research question:

> *Can a very small diffusion model, whose denoiser and text encoder are both trained from scratch, generalize to combinations of avatar attributes that were not observed together during training?*

Evaluating this question requires going beyond standard in-distribution validation to test **compositional out-of-distribution (OOD) generalization**. Specifically, the model must synthesize avatars conditioned on unseen pairings of known individual attributes (e.g., combining "yellow hair" with "sunglasses" when these two features were strictly isolated during training).

C. Key Contributions

To answer this research question under strict from-scratch constraints, our main contributions are as follows:

1) **End-to-End From-Scratch Implementation:** We have designed and implemented a complete pixel-space Denoising Diffusion Probabilistic Model (DDPM) pipeline using native PyTorch abstractions. The system comprises a custom deterministic text Tokenizer, a Text Encoder, a compact U-Net denoiser incorporating spatial cross-attention layers and a reverse DDPM sampling loop. Each module is completely independent of pretrained weights or high-level black-box libraries.
2) **Compositional OOD Benchmark Protocol:** We use a precise data preprocessing and partitioning strategy on the Google Cartoon Set [4]. By holding out specific attribute combinations while ensuring vocabulary construction uses only training split data, we construct dedicated *Ordinary Test* (in-distribution) and *Compositional-OOD Test* evaluation splits to measure new attributes recombinations.
3) **Empirical and Efficiency Evaluation:** We conduct quantitative and qualitative evaluations. We measure image quality using Fréchet Inception Distance (FID)

and Kernel Inception Distance (KID). We also evaluate sampling diversity across random seeds and profile hardware efficiency metrics, such as parameter counts, sampling throughput and peak VRAM allocation on the experimental hardware.

II. Related Work

A. Denoising Diffusion Probabilistic Models

Denoising Diffusion Probabilistic Models (DDPMs) [2] synthesize images by learning to iteratively reverse a forward Gaussian corruption process. Unlike Latent Diffusion Models (LDMs) [3] that perform denoising within a compressed latent space, pixel-space DDPMs operate directly on raw spatial pixel representations.

B. Text-Conditioned Generation and Cross-Attention

Guiding image synthesis via natural language descriptions requires integrating textual semantics into the visual denoising network. Spatial cross-attention mechanisms, which were first introduced in the Transformer architecture [5], enable intermediate image feature maps to consider contextual token representations [3]. Modern text-to-image systems usually depend on large, pretrained language models (e.g., CLIP [6] or T5 [7]) to extract textual features.

C. Compositional OOD Generalization

Evaluating generative models beyond standard in-distribution validation is essential to measure true concept recombination rather than memorization. Compositional Out-of-Distribution (OOD) evaluation tasks the model with synthesizing unseen attribute combinations (e.g., combining specific colours with distinct artworks that were strictly separated during training).

III. Dataset and Compositional Split

A. Data Collection and Inspection

Starting with the dataset used for this project, we selected the Google Cartoon Set [4], which is a collection of randomly generated 2D cartoon avatars. The Google Cartoon set consists of two sets of 10,000 and 100,000 500×500 RGBA PNG avatar images, respectively. Avatars vary in 10 artwork categories, 4 colour categories and 4 proportion categories, as shown in the Table V. In addition, each avatar image is associated with a CSV file that contains attributes for the cartoon. Each row in the attribute file states the name of the attribute, the selected variant index, and the total number of variants of the attribute.

B. Data Preprocessing

The images in the dataset were scaled from 500×500 to 32×32 and 64×64 to create the sub-sets to use in the experiments. This was achieved by first cropping the images on each side by 64 pixels and then scaling them by bilinear interpolation. Cropping has been used to reduce the unused white space around the avatars to a minimum and to increase the informative pixels when the images are downsampled. Finally, the pixel channel values of the images were normalized from $[0, 255]$ to $[-1, 1]$ by using the module `torchvision.transforms.Normalize()` with a mean of 0.5 and a standard deviation of 0.5.

C. Caption Generation and Vocabulary Construction

From the CSV files associated with each image, an artificial deterministic mapping was created by comparing the image's artwork, colour or proportion variant index with the real image and then translating it into a short descriptive string of words. Deterministic captions were then generated for each image by filling the common caption template with words that describe the avatar, in the order and mapping described in the Table V, and then saved in TXT files.

Subsequently, after the captions were generated, the tokenizer was built from scratch. To build the vocabulary, the tokenizer reads each caption, converts it to lower-case, removes all the punctuation, and registers each new word with a unique id, ignoring empty words and blank spaces. Moreover, five special tokens were added to the vocabulary:

- `[PAD]` padding token, to equalize the length of captions.
- `[UNK]` unknown token, for words not in the vocabulary.
- `[SOS]` start of sequence token.
- `[EOS]` end of sequence token.

D. Compositional Out-Of-Distribution Split

To evaluate the model's ability to perform systematic concept recombination rather than memorizing training samples, we construct a dedicated Compositional Out-Of-Distribution (OOD) benchmark protocol. Standard random image-level splitting only measures in-distribution generalization, as test avatars frequently share feature combinations present during training. In contrast, our compositional evaluation explicitly tests whether the model can synthesize images conditioned on novel pairings of individual visual attributes that were never observed together in the training set.

Specifically, we hold out four distinct combinations across key visual dimensions:

1) `hair_color` (*black*) + `facial_hair` (*mustache with goatee*);
2) `hair_color` (*red*) + `eye_color` (*blue*);
3) `face_color` (*fair pinkish*) + `hair` (*top knot bun*);
4) `glasses` (*cat eye dark sunglasses*) + `hair_color` (*yellow*).

All avatar images containing any of these four exact attribute combinations are completely removed from both the training and validation sets and assigned exclusively to the *Compositional-OOD Test* set ($N_{\text{OOD}} = 295$). To ensure that the model has fully learned the underlying visual semantics of each constituent attribute independently, we guarantee that all individual values (e.g., *red hair* or *blue eyes*) appear with a variable frequency (N_{tr1} or N_{tr2}) alongside other attributes in the training split. As detailed in Table I, the resulting dataset partitioning consists of $8,734$ training images, 971 validation images, and 295 compositional OOD test images for both 32×32 and 64×64 resolution subsets, contrasting with

the $8,000/1,000/1,000$ split used in the standard Ordinary baseline.

TABLE I
DATASET PARTITIONING STATISTICS AND COMPOSITIONAL OOD HELD-OUT COMBINATIONS

Split Name	Split Type	Resolution	Train	Validation	Test
`ordinary_32`	Ordinary	32×32	8,000	1,000	1,000
`ordinary_64`	Ordinary	64×64	8,000	1,000	1,000
`compositional_32`	Compositional	32×32	8,734	971	295
`compositional_64`	Compositional	64×64	8,734	971	295

Compositional Held-Out Combinations (OOD Test)			
Attribute Pair (Value 1 + Value 2)	N_{OOD}	N_{tr1}	N_{tr2}
`hair_colour` (*black*) + `facial_hair` (*moustache w/ goatee*)	33	885	286
`hair_colour` (*red*) + `eye_colour` (*blue*)	203	702	1,634
`face_colour` (*fair pinkish*) + `hair` (*top knot bun*)	12	773	67
`glasses` (*cat eye dark ...*) + `hair_colour` (*yellow*)	47	337	866

IV. METHODOLOGY AND ARCHITECTURE

This section explains the implementation of each module. The entire architecture was built from scratch with PyTorch [8] modules and is summarized in Figure 1.

A. Text Encoder

The Text Encoder takes an encoded caption as input and processes it to produce a latent representation that can be used as a conditioning mechanism.

1) **Token and Positional Embedding:** Given an input token sequence $x \in \mathbb{R}^{B\times S}$, with B the batch size and S the length of the sequence, tokens are mapped to dense embeddings $e_{tok} \in \mathbb{R}^{B\times S\times D}$, with D the embedding dimension, using `torch.nn.Embedding()`. Sinusoidal positional encodings $e_{pos} \in \mathbb{R}^{S\times D}$ are added to retain sequence order, yielding after dropout (DO):
$$z_0 = \mathrm{DO}(e_{tok} + e_{pos}) \in \mathbb{R}^{B\times S\times D} \tag{1}$$
2) **Transformer Encoder Blocks:** z_0 is passed through E stacked encoder layers. Each layer consists of:
   - *Multi-Head Self-Attention (MHSA):* Computes contextual relations across all tokens with A heads using `torch.nn.MultiheadAttention()`.
   - *Feed-Forward Network (FFN):* A two-layer MLP with SiLU activation and dropout.
   - *Dropout (DO), Residual Connections & LayerNorm (LN):* Applied after both MHSA and FFN:
$$z'_l = \mathrm{LN}(z_{l-1} + \mathrm{DO}(\mathrm{MHSA}(z_{l-1}))) \tag{2}$$
$$z_l = \mathrm{LN}(z'_l + \mathrm{DO}(\mathrm{FFN}(z'_l))) \tag{3}$$
3) **Output:** The final layer representation $z_N \in \mathbb{R}^{B\times L\times D}$ will be used as the text sequence embeddings $c_{text} \in \mathbb{R}^{B\times L\times D}$ for conditioning the U-Net.

B. U-Net Denoiser

The baseline unconditional U-Net predicts the added noise ϵ given the noisy pixel representation $x_t \in \mathbb{R}^{B\times C\times H_0\times W_0}$, with C the input and output channels of the model (3 for RGB) and H_0, W_0 the input and output image dimensions, and the time step index t. The U-Net operates through the following steps:

1) **Input Convolution:** The input noisy pixel representation x_t is passed through an initial convolutional layer ($\mathrm{CL}_{0,\text{in}}$) to project it to the base channels C_0. Each convolutional layer used in the architecture has a 3×3 kernel, 1-unit padding and a stride of 1 in order to preserve the dimensions of the input:
$$z_0 = \mathrm{CL}_0(x_t) \in \mathbb{R}^{B\times C_0\times H_0\times W_0} \tag{4}$$
2) **Time Step Embedding:** The scalar time step t is mapped into a sinusoidal positional encoding $e_{pos} \in \mathbb{R}^{B\times C_0}$ and then processed via a two-layer MLP to produce the time embedding $e_t \in \mathbb{R}^{B\times D_{time}}$, with $D_{time} = K \cdot C_0$ and $K = 4$.
3) **Encoder Path (Downsampling):** The input projection z_0 is passed through a sequence of L convolutional Residual blocks (RB) and Downsampling layers. At each resolution stage $l = 0, \ldots, L$, spatial dimensions are reduced ($H_{l+1}, W_{l+1} \leftarrow H_l/2, W_l/2$) while channel capacity is expanded ($C_{l+1} \leftarrow 2C_l$):
   - The input feature map $z_{l-1} \in \mathbb{R}^{B\times C_{l-1}\times H_{l-1}\times W_{l-1}}$ is passed through a first chain of Group Normalization (GN) [9] $\rightarrow$ SiLU $\rightarrow$ Convolution ($\mathrm{CL}_{\text{l,in}}$):
$$h_l = \mathrm{CL}_{\text{l,in}}(\mathrm{SiLU}(\mathrm{GN}(z_{l-1}))) \in \mathbb{R}^{B\times C_l\times H_{l-1}\times W_{l-1}} \tag{5}$$
   - The time embedding e_t is projected and reshaped to $(B, C_l, 1, 1)$, and added via spatial broadcasting to the intermediate feature map h_l:
$$h_{l,t} = h_l + \mathrm{Linear}(e_t) \tag{6}$$
   - The intermediate feature map h_l is passed through a second chain of GN $\rightarrow$ SiLU $\rightarrow$ DO $\rightarrow$ $\mathrm{C}_{\text{l,out}}$:
$$z'_l = \mathrm{CL}_{\text{l,out}}(\mathrm{DO}(\mathrm{SiLU}(\mathrm{GN}(h_{l,t})))) \in \mathbb{R}^{B\times C_l\times H_{l-1}\times W_{l-1}} \tag{7}$$
   - The output feature map z_l is obtained by downsampling z'_l via a convolutional layer (DS) with a 3×3 kernel, 1-unit padding and a stride of 2:
$$z_l = \mathrm{DS}(z'_l) \in \mathbb{R}^{B\times C_l\times H_l\times W_l} \tag{8}$$
4) **Bottleneck Layer:** At the lowest spatial resolution L, $z_{L,in}$ is passed through a bottleneck layer composed of a Residual block, an Attention block (AB, explained in the following subsection) and another Residual block. This layer does not expand channel capacity, but helps to capture the global visual context before upsampling:
$$z_{L,out} = \mathrm{RB}(\mathrm{AB}(\mathrm{RB}(z_{L,in}))) \in \mathbb{R}^{B\times C_l\times H_l\times W_l} \tag{9}$$
5) **Decoder Path (Upsampling and Skip Connections):** The bottleneck output feature map $z_{L,out}$ is progressively upsampled back to the original resolution ($H \times$

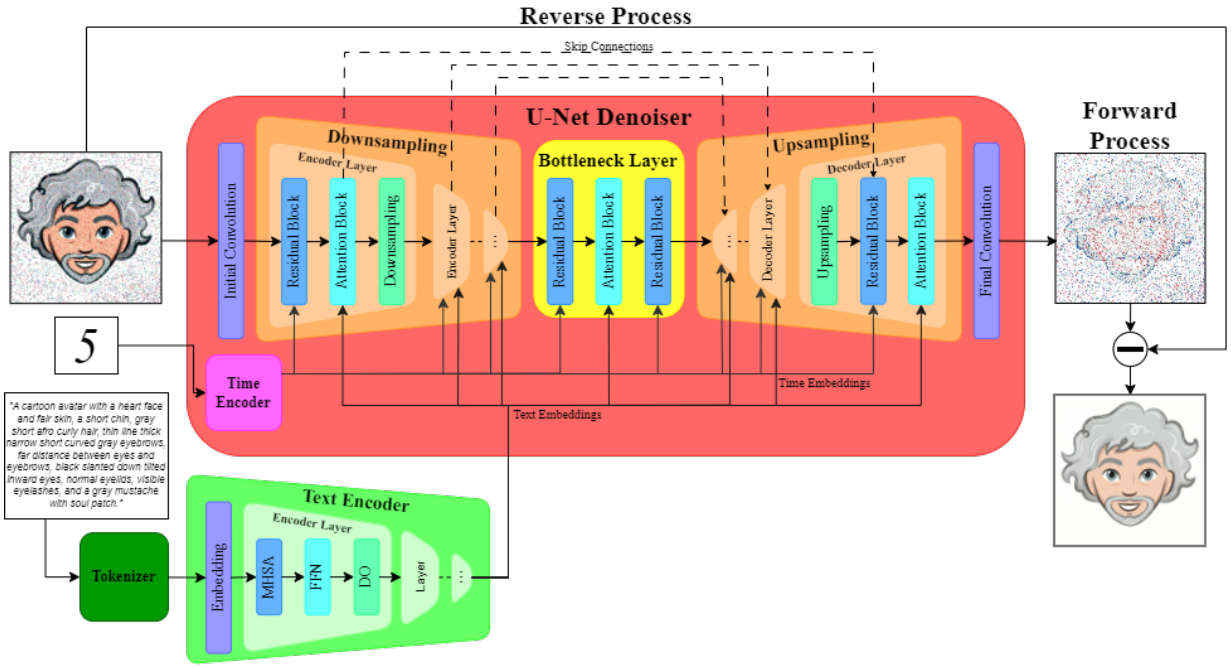


Fig. 1. Architecture of the proposed models. In the Forward Process the U-Net takes as input the noisy sample and the relative time step and outputs the predicted noise. The Tokenizer and Text Encoder take the caption and produce the Text Embeddings that are fed to the Attention Blocks. In order to reconstruct the image during the Reverse Process, the predicted noise is subtracted from the noisy sample.

W). At each resolution stage $l = L, \dots, 0$, spatial dimensions are expanded ($H_{l-1}, W_{l-1} \leftarrow 2H_l, 2W_l$) while channel capacity is reduced ($C_{l-1} \leftarrow C_l/2$):

- Spatial resolution of z_l is doubled using nearest-neighbour interpolation (NNI) with a scale factor of 2, followed by a convolutional layer (US) with a 3×3 kernel, 1-unit padding and a stride of 1:

$$z'_l = \mathrm{UP}(\mathrm{NNI}(z_l)) \in \mathbb{R}^{B \times C_l \times H_{l-1} \times W_{l-1}} \quad (10)$$

- The corresponding encoder feature map $z'_{l,e}$ is retrieved and concatenated along the channel dimension with the upsampled feature map z'_l:

$$z'_{l,cat} = \mathrm{Cat}(z'_l, z'_{l,e}) \in \mathbb{R}^{B \times 2C_l \times H_{l-1} \times W_{l-1}} \quad (11)$$

- The concatenated feature map is processed by a Residual block with time embedding e_t to obtain the output feature map z_{l-1}:

$$z_{l-1} = \mathrm{RB}(z'_{l,cat}, e_t) \in \mathbb{R}^{B \times C_{l-1} \times H_{l-1} \times W_{l-1}} \quad (12)$$

6) **Output Convolution:** A final chain of GN $\rightarrow$ SiLU $\rightarrow$ $\mathrm{CL}_{0,\mathrm{out}}$ maps the channel dimension back to C (RGB space), outputting the noise estimate $\hat{\epsilon}(x_t, t)$:

$$\hat{\epsilon}(x_t, t) = \mathrm{CL}_{0,\mathrm{out}}(\mathrm{SiLU}(\mathrm{GN}(z_0))) \in \mathbb{R}^{B \times C \times H \times W} \quad (13)$$

C. Conditioning Mechanism

To guide the denoising process, the U-Net incorporates the sequence of textual embeddings c_{text} produced by the Text Encoder. This is executed by an Attention block (AB) in each resolution stage l, between the Residual block and the Downsampling/Upsampling layer:

1) **Self-Attention:** The output of the Residual block z'_l is passed through a GN, a convolutional layer $\mathrm{CL}_{\mathrm{l,in}}$ (with a 1×1 kernel, 0-units padding and a stride of 1) and then reshaped to $(B, H_{l-1} \cdot W_{l-1}, C_l)$. This feature map is then passed through a LayerNorm (LN) and a Multi-Head Self-Attention (MHSA) layer with A heads, which captures global spatial relationships. Finally, a residual connection is applied:

$$h_{l,in} = \mathrm{CL}_{\mathrm{l,in}}(\mathrm{GN}(z'_l)) \in \mathbb{R}^{B \times C_l \times H_{l-1} \times W_{l-1}} \quad (14)$$

$$h_{l,in} \rightarrow h'_{l,in} \in \mathbb{R}^{B \times (H_{l-1} \cdot W_{l-1}) \times C_l} \quad (15)$$

$$h_{l,s} = h'_{l,in} + \mathrm{MHSA}(\mathrm{LN}(h'_{l,in})) \in \mathbb{R}^{B \times (H_{l-1} \cdot W_{l-1}) \times C_l} \quad (16)$$

2) **Cross-Attention:** The self-attention output $h_{l,s}$ is passed through a second LayerNorm (LN) and then combined with the textual embeddings c_{text} via a Multi-Head Cross-Attention (MHCA) layer with A heads. This layer measures the correlation between spatial information and text tokens. Finally, a residual connection is applied:

$$h_{l,c} = h_{l,s} + \mathrm{MHCA}(\mathrm{LN}(h_{l,s}), c_{text}) \in \mathbb{R}^{B \times (H_{l-1} \cdot W_{l-1}) \times C_l} \quad (17)$$

3) **Feed-Forward Network:** The cross-attention output $h_{l,c}$ is passed through a third LayerNorm (LN) and a feed-forward network (FFN) with SiLU activation. This projects the feature map into a higher-dimensional space, $\mathbb{R}^{B\times(H_{l-1}\cdot W_{l-1})\times(K\cdot C_l)}$, with $K = 4$, and then back into the original space, $\mathbb{R}^{B\times(H_{l-1}\cdot W_{l-1})\times C_l}$. Finally, a residual connection is applied:

$$h_{l,n} = h_{l,c} + \mathrm{FFN}(\mathrm{LN}(h_{l,c})) \in \mathbb{R}^{B\times(H_{l-1}\cdot W_{l-1})\times C_l} \tag{18}$$

4) **Output Convolution:** The feed-forward network output $h_{l,n}$ is reshaped to its original shape $(B, C_l, H_{l-1}, W_{l-1})$ and passed through a convolutional layer $\mathrm{CL}_{\mathrm{l,out}}$ with a 1×1 kernel, 0-units padding and a stride of 1. Finally, a residual connection with the input of the Attention block is applied. This will be the input for the subsequent Downsampling/Upsampling layer:

$$h_{l,n} \to h'_{l,n} \in \mathbb{R}^{B\times C_l\times H_{l-1}\times W_{l-1}} \tag{19}$$

$$z''_l = z'_l + \mathrm{CL}_{\mathrm{l,out}}(h_{l,n}) \in \mathbb{R}^{B\times C_l\times H_{l-1}\times W_{l-1}} \tag{20}$$

D. Diffusion Process and Noise Scheduler

Denoising Diffusion Probabilistic Models (DDPM) formulate image generation through a dual-process framework: a forward noise-addition process q and a parametrized reverse denoising process p_θ [2]:

1) **Forward Diffusion Process:** The forward process gradually corrupts a clean image $x_0 \sim p_{\text{data}}$ by adding Gaussian noise over a sequence of T discrete time steps $t \in \{1,\dots,T\}$. At each step t, conditional noise is injected according to a predefined variance schedule $\beta_t \in (0,1)$:

$$q(x_{1:T} \mid x_0) := \prod_{t=1}^{T} q(x_t \mid x_{t-1}) \tag{21}$$

$$q(x_t \mid x_{t-1}) := \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1},\, \beta_t \mathbf{I}\right) \tag{22}$$

By defining $\alpha_t = 1-\beta_t$ and its cumulative product $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, x_t can be sampled in closed form directly from x_0 at any arbitrary time step t without iterating:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{23}$$

As $t \to T$, $\bar{\alpha}_T \to 0$, causing x_T to converge to pure Gaussian noise $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

2) **Noise Schedule Strategy:** The choice of variance schedule β_t controls the rate at which Gaussian noise corrupts the image features across time $t \in \{1,\dots,T\}$. To evaluate the sensitivity of denoising dynamics, we explore both linear and cosine variance schedules:

- **Scaled Linear Variance Schedule:** We define a scaled linear schedule where β_t interpolates linearly in the square root space between $\beta_{\text{start}} = 0.0001$ and $\beta_{\text{end}} = 0.02$ over $T = 1000$ time steps. This approach produces very small values at early time steps, followed by progressively larger values at later time steps. This graduated increase ensures smooth noise addition throughout the diffusion process:

$$\beta_t = \left(\sqrt{\beta_{\text{start}}} + \frac{t-1}{T-1}\left(\sqrt{\beta_{\text{end}}} - \sqrt{\beta_{\text{start}}}\right)\right)^2 \tag{24}$$

- **Cosine Variance Schedule:** To prevent premature destruction of fine-grained spatial details at small time steps, we also adopt a cosine noise schedule [10], which enforces a smoother transition:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{\frac{t}{T}+s}{1+s}\cdot\frac{\pi}{2}\right)^2 \tag{25}$$

where $s = 0.008$ is a small offset parameter used to prevent β_t from being too small near $t = 0$. The step variance is then bounded by $\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}$ and clipped to $\beta_{\text{max}} = 0.999$.

3) **Reverse Process and Training Objective:** The reverse process aims to reconstruct x_0 by iteratively removing noise $p_\theta(x_{t-1} \mid x_t, c_{\text{text}})$. The U-Net denoiser $\epsilon_\theta(x_t, t, c_{\text{text}})$ is trained to predict the added Gaussian noise ϵ by minimizing the mean-squared error (MSE) loss function:

$$\mathcal{L}_{\text{DDPM}}(\theta) := \mathbb{E}_{t,x_0,\epsilon}\left[\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon,\, t,\, c_{\text{text}}\right)\right\|^2\right] \tag{26}$$

4) **Reverse Sampling Algorithm** During inference, starting from pure noise $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the model iteratively computes x_{t-1} for $t = T,\dots,1$ using the predicted noise:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t, c_{\text{text}})\right) + \sigma_t z \tag{27}$$

where $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $t > 1$ ($z = \mathbf{0}$ when $t = 1$), and $\sigma_t^2 = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$ represents the posterior variance.

E. Code Availability

The complete code with requirements and instructions for reproducibility is available, without checkpoints, at the GitHub repository [1] or, complete with checkpoints, at the OneDrive share point [11].

V. Experimental Setup

To ensure full reproducibility and evaluation, all experiments are conducted using `Python 3.14.7` and `PyTorch 2.14.1` within a standardized hardware environment. This section details the baseline configurations, optimization hyperparameter choices, and the computational profiling metrics employed across model variants.

A. Model Configurations and Baselines

To isolate the generative contribution of textual conditioning, we design and compare two primary network configurations sharing an identical core architecture:

1) **Unconditional Neural Baseline:** To establish a baseline for unconditional visual generation without altering

model capacity or network architecture, we condition the U-Net on a null caption sequence comprising only control tokens (`prompt = [SOS], [EOS], [PAD], ...`). The Text Encoder processes this empty input into a neutral context representation c_{null}, effectively compelling the U-Net cross-attention layers to operate without attribute guidance.

2) **Text-Conditioned Model:** The proposed conditional model, which dynamically injects multi-attribute captions representations extracted by the Text Encoder directly into the U-Net via spatial cross-attention layers. The encoder is jointly trained from scratch with the U-Net.

B. Training Protocol and Hyperparameters

Both models are trained in pixel space to minimize the mean squared error (MSE) between the added noise ϵ and the U-Net estimate $\epsilon_\theta(x_t, t, c_{\text{text}})$.

- **Optimization and Learning Rate:** We employ the `torch.optim.AdamW` optimizer with an initial learning rate of $\eta_0 = 2{\cdot}10^{-4}$ and a weight decay $\omega = 10^{-2}$. To stabilize optimization, we apply a dynamic learning rate schedule consisting of a linear warm-up phase followed by a cosine annealing decay with a non-zero floor ratio:

$$\eta(t) = \begin{cases} \eta_0 \cdot \frac{t+1}{T_{\text{warmup}}}, & \text{if } t < T_{\text{warmup}} \\ \eta_0 \cdot \left[\gamma + (1-\gamma) \cdot \frac{1}{2}\left(1 + \cos\left(\pi \frac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}}\right)\right)\right], & else \end{cases} \tag{28}$$

  where t denotes the current optimization step, $T_{\text{warmup}} = 100$ is the number of linear warm-up steps, T_{total} is the total number of training steps across all epochs, and $\gamma = 0.05$ represents the minimum learning rate ratio ($\eta_{\text{min}} = \gamma \cdot \eta_0 = 10^{-5}$).
- **Noise Schedule Setup:** Forward diffusion spans $T = 1,000$ discrete time steps under both scaled linear ($\beta_{\text{start}} = 0.0001$, $\beta_{\text{end}} = 0.02$) and cosine ($\beta_{\text{max}} = 0.999$, $s = 0.008$) variance schedules.
- **Validation:** After each epoch, the model is validated by sampling ten equally spaced time steps from the noise scheduler. The ten noisy input samples for each image in the validation set are then passed to the model. The final validation loss is computed as the MSE.
- **Classifier-Free Guidance:** During the training of conditioned models, a dropout rate of 10% is applied to the captions in order to train part of the model as an unconditioned one. This enables Classifier-Free Guidance [12] to be applied with a scale of $w = 3.5$ during inference, enhancing diversity across multiple generations with different seeds.
- **Reproducibility and Checkpointing:** Experiments are executed under a fixed global random seed $= 42$. Model checkpoints, optimizer states and validation loss histories are cached periodically to enable deterministic inference and training resumption.

C. Hardware and Computational Metrics

All experiments are executed on an NVIDIA GeForce RTX 3070 Laptop GPU (8 GB VRAM) with `CUDA UMD Version 13.4`. To assess practical efficiency alongside generative quality, we profile three computational metrics:

1) **Parameter Count:** Detailed reporting of learnable parameters for both the compact U-Net denoiser and the Text Encoder.
2) **Sampling Time:** Average latency required to generate batches of different sizes of 32×32 or 64×64 avatar images across $T = 250$ reverse DDPM steps.
3) **GPU VRAM Peak Allocation:** Peak memory consumption recorded via `torch.cuda.max_memory_allocated()` during reverse sampling inference loops.

VI. Results and Evaluation

This section provides a quantitative and qualitative evaluation of our pixel-space DDPM. It compares the unconditional neural baseline with the full text-conditioned model, examining both in-distribution and compositional out-of-distribution (OOD) test splits. We evaluated a total of eight model architectures and hyperparameters configurations described in Table II. The results of the experiments are summarized in Table III.

A. Quantitative Image Quality

To assess visual fidelity and feature alignment, we evaluate Fréchet Inception Distance (FID) and Kernel Inception Distance (KID) across 250 generated samples per configuration:

1) **Fréchet Inception Distance (FID):** FID models the feature distributions of real (x_r) and generated (x_g) images as multi-dimensional Gaussians $\mathcal{N}(\mu_r, \Sigma_r)$ and $\mathcal{N}(\mu_g, \Sigma_g)$ [13]. The feature representations are obtained from the final average pooling layer of the Inception-v3 model [14]. The FID score is calculated as the squared Wasserstein-2 distance [15] between these two Gaussian distributions:

$$\text{FID}(r, g) = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\right) \tag{29}$$

   where μ_r, μ_g are the mean feature vectors, Σ_r, Σ_g are the feature covariance matrices, and $\text{Tr}(\cdot)$ denotes the matrix trace. Lower FID values indicate greater visual quality and higher distribution similarity to real training avatars.
2) **Kernel Inception Distance (KID):** While the FID provides a robust global measure, its estimation can be severely biased when computed over small sample sizes, often overestimating the true distance [16]. To mitigate this, we compute the Kernel Inception Distance (KID) [17], which measures the maximum mean discrepancy (MMD) between real and generated Inception-v3 feature distributions using a polynomial kernel function.

$$k(x, y) = \left(\frac{1}{d} x^T y + 1\right)^3 \tag{30}$$

TABLE II
MODEL ARCHITECTURES AND HYPERPARAMETER CONFIGURATIONS

Model Configuration	$H \times W$	C_0	L	A	D_{time}	S	E	D_{text}	DO	Epochs	B	η_0	ω	Schedule Strategy
unconditioned-32	32×32	64	3	4	256	70	4	128	0.1	50	32	$2 \cdot 10^{-4}$	10^{-2}	Scaled Linear
conditioned-ordinary-32	32×32	64	3	4	256	70	4	128	0.1	50	32	$2 \cdot 10^{-4}$	10^{-2}	Scaled Linear
compositional-linear-32	32×32	**128**	3	4	**512**	70	4	128	0.1	50	32	$2 \cdot 10^{-4}$	10^{-2}	Scaled Linear
compositional-cosine-32	32×32	64	3	4	256	70	4	128	0.1	50	32	$2 \cdot 10^{-4}$	10^{-2}	Cosine
unconditioned-64	64×64	128	3	4	512	70	4	128	0.1	25	10	$2 \cdot 10^{-4}$	10^{-2}	Scaled Linear
conditioned-ordinary-64	64×64	128	3	4	512	70	4	128	0.1	25	10	$2 \cdot 10^{-4}$	10^{-2}	Scaled Linear
compositional-linear-64	64×64	**64**	3	4	**256**	70	4	128	0.1	25	10	$2 \cdot 10^{-4}$	10^{-2}	Scaled Linear
compositional-cosine-64	64×64	128	3	4	512	70	4	128	0.1	25	10	$2 \cdot 10^{-4}$	10^{-2}	Cosine

TABLE III
QUANTITATIVE PERFORMANCE, EFFICIENCY, AND RESOURCE PROFILING SUMMARY

Model Configuration	**Evaluated Split**	**Parameters**	**Size** (MB)	**FID**	**KID** ($\times 10^{-3}$)	**Diversity Score** ($\times 10^{-3}$)
unconditioned-32	ordinary_32	37,782,787	144.12	50.8532	3.738 ± 4.011	82.020
conditioned-ordinary-32	ordinary_32	37,782,787	144.12	52.3479	1.481 ± 2.487	0.047
compositional-linear-32	compositional_32	147,603,459	563.06	23.7674	-1.480 ± 2.147	0.037
compositional-cosine-32	compositional_32	37,782,787	144.12	27.9363	-1.029 ± 2.669	6.267
unconditioned-64	ordinary_64	147,603,331	563.06	31.3289	3.216 ± 2.289	96.902
conditioned-ordinary-64	ordinary_64	147,603,331	563.06	28.9332	0.867 ± 1.814	3.410
compositional-linear-64	compositional_64	37,782,659	144.12	21.0904	1.233 ± 1.740	6.761
compositional-cosine-64	compositional_64	147,603,331	563.06	14.1279	-0.931 ± 1.489	0.181

where $d = 2048$ represents the feature dimension. Unlike FID, KID is an unbiased estimator that does not assume a parametric Gaussian distribution. During evaluation, random sub-batches of real and generated images are repeatedly drawn to compute the MMD per subset. Consequently, we report both the sample mean (μ_{KID}) and standard deviation (σ_{KID}) across subsets.

3) **Implementation Details via PyTorch Metrics:** Following standard evaluation guidelines, our metric computation pipeline is implemented using native PyTorch abstractions via the `torchmetrics.image.fid` and `torchmetrics.image.kid` modules [18].
   - **Preprocessing and Rescaling:** Generated images ($x_g \in [-1, 1]$) and ground-truth images are rescaled to $[0, 255]$, and converted to 8-bit unsigned integer tensors (`torch.uint8`) of shape $(B, 3, H, W)$ as required by the PyTorch Inception-v3 feature extractor.
   - **Inception Feature Extraction:** Features are extracted from the default 2048-dimensional pooling layer of Inception-v3.
   - **Batch Update and Computation:** Metric state representations are updated incrementally via `fid.update(real_imgs, real=True)` and `fid.update(gen_imgs, real=False)`. The final score is computed over 250 samples per evaluation split (*Ordinary Test* vs. *Compositional-OOD Test*), with a subset size of 50 for KID.

B. Compositional OOD Generalization

To answer our primary research question, we evaluate the model on the *Compositional-OOD Test Split*, which contains attribute combinations that were strictly held out during training (Table I). For each of the four combinations considered, we randomly select a ground truth avatar from the test set and ask each model to generate an avatar with that specific caption:

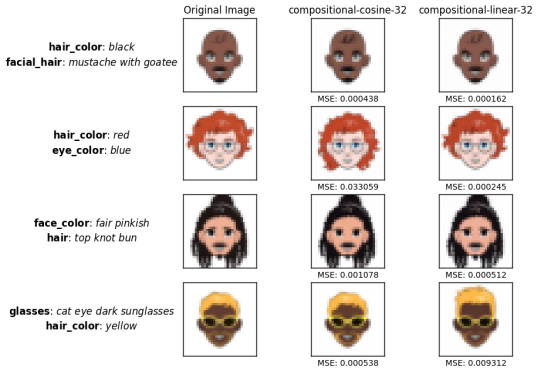
- **Qualitative Evaluation:** Figure 2 shows a grid of the generated images for each held-out combination, compared with the ground truth example sampled from the test set.
- **Quantitative Evaluation:** The pixel-to-pixel mean squared error (MSE) is computed for each generated image with respect to the ground truth example. The lower the error, the closer the generated image is to the expected result when, taking into account the diversity introduced by the Classifier-Free Guidance.

C. Diversity and Seed Sensitivity

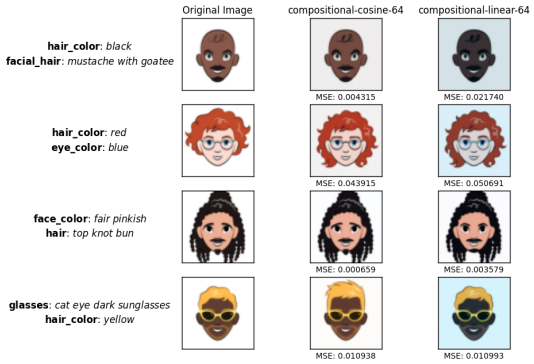
Starting from a `numpy.random` generator with a fixed base seed = 210503, we evaluate image diversity across seeds for the same prompt. This is done by sampling a random caption from the test set and generating four images for each model with a random seed from the numpy generator. Moreover, we compute the mean squared error (MSE) between each pair of images. The lower the error, the closer the images are, even when different seeds are used, and the weaker the model is at generating a novel result. The error is labelled as *Diversity Score* in Table III and the images are displayed in Figure 3.

D. Computational Efficiency

The sampling time per image and the peak VRAM memory consumption across batches of different sizes are presented in Table IV and displayed in Figure 4 and Figure 5 respectively.

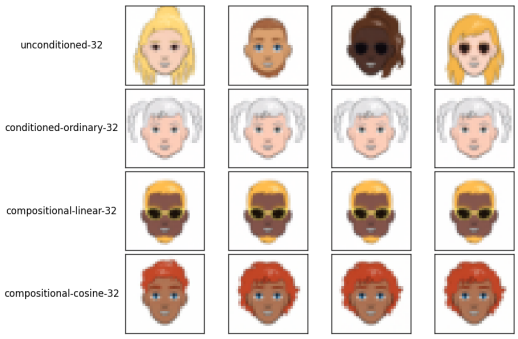


(a). Test results for compositional models 32×32.

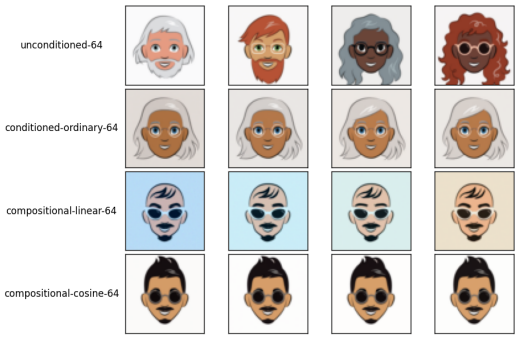


(b) Test results for compositional models 64×64.

Fig. 2. Sample generations of OOD combinations (on the left) from models trained on compositional splits compared with the expected originals. Under each sample, the MSE between the generated and the original is reported.



(a). Sample generations from models 32×32.



(b) Sample generations from models 64×64.

Fig. 3. Sample generations on the same prompt across different seeds.

TABLE IV
SAMPLING TIME AND PEAK VRAM USAGE PER BATCH SIZE

Batch Size:	1	2	4	8	16	32
Model Configuration	**Sampling Time** (s/img)					
unconditioned-32	11.2781	5.7298	2.9500	1.4926	1.1407	0.9736
conditioned-ordinary-32	12.0549	5.9134	2.8751	1.4546	1.0380	0.9489
compositional-linear-32	11.3121	5.6786	3.2869	2.8429	2.6434	2.1889
compositional-cosine-32	11.4941	5.6370	2.7467	1.4233	1.0368	0.9772
unconditioned-64	18.9257	17.8852	16.6111	16.1922	15.9116	14.8583
conditioned-ordinary-64	19.0143	17.0479	15.9268	15.9014	15.4755	13.6288
compositional-linear-64	12.1489	9.8134	9.0851	8.4202	8.2257	7.3467
compositional-cosine-64	18.8560	16.7454	15.6279	15.8473	15.6405	14.4413
Model Configuration	**Peak VRAM Usage** (MB)					
unconditioned-32	604.20	608.01	629.29	675.87	769.93	955.29
conditioned-ordinary-32	604.20	608.01	629.29	675.87	769.93	955.29
compositional-linear-32	2344.41	2351.46	2365.26	2450.59	2635.76	3006.06
compositional-cosine-32	604.20	608.01	629.29	675.87	769.93	955.29
unconditioned-64	2372.90	2450.14	2634.87	3004.31	3745.17	5222.12
conditioned-ordinary-64	2372.90	2450.14	2634.87	3004.31	3745.17	5222.12
compositional-linear-64	629.44	675.43	768.15	953.59	1326.38	2067.54
compositional-cosine-64	2372.90	2450.14	2634.87	3004.31	3745.17	5222.12

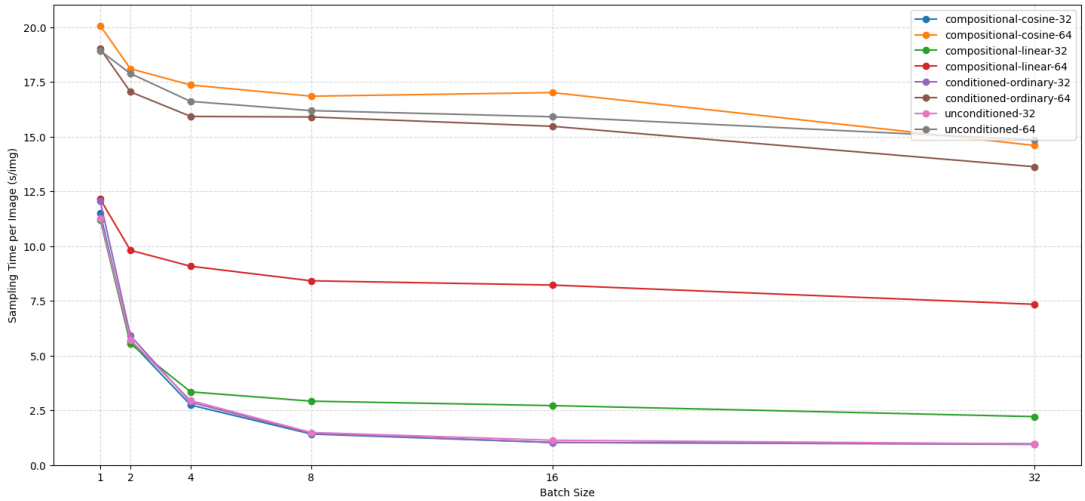


Fig. 4. Sampling Time per Image vs Batch Size

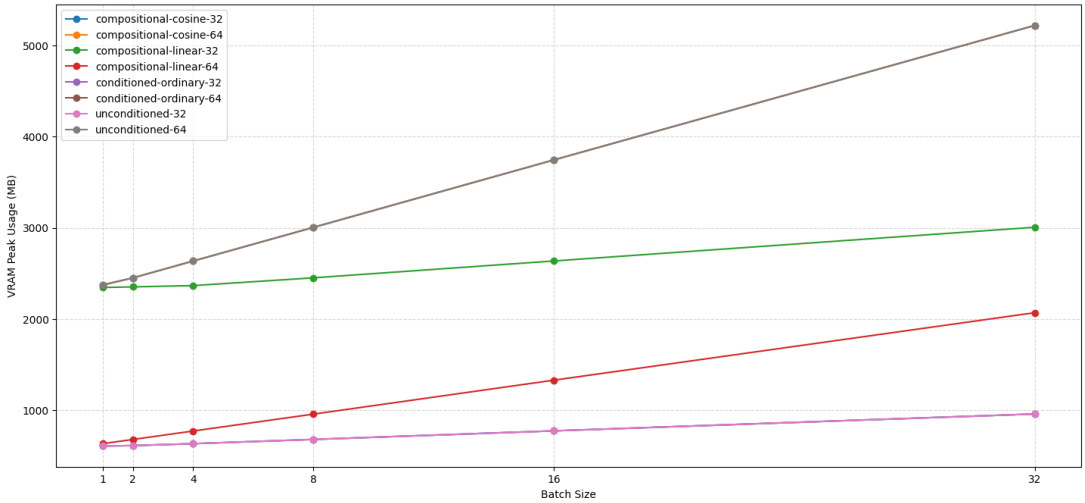


Fig. 5. VRAM Peak Usage vs Batch Size

VII. Discussion and Challenges

This section summarizes our empirical findings, addresses the primary research question concerning new attribute combinations, outlines the technical challenges encountered during development and offers recommendations for future research.

A. Analysis of Findings and Research Question Answer

Our central research question inquired whether a small pixel-space diffusion model, equipped with a denoiser and text encoder trained entirely from scratch, could generalize to unseen combinations of avatar attributes.

1) **Quantitative Image Quality:** As shown in Table III, the FID and KID values indicate that the trained models can generate images that are close to the real distribution. However, the higher values of the FID confirm that the chosen metric is not suitable for small pools of 250 samples, as has been reported in the literature [16].
KID values tend to be lower than FID values because they are not affected by the number of samples used for estimation. In some cases, such as for *compositional-linear-32*, *compositional-cosine-32* and *compositional-cosine-64*, the reported KID mean value is negative, even though positive values fall within the confidence interval of the standard deviation. Negative KID mean values have already been reported in the literature [19], due to the index's large variance. Using different metrics could correct this behaviour. However, the fact that all the measured KID values are close to zero still confirms that the models can generate images close to the real distribution.
2) **Compositional OOD Generalization:** Qualitative evaluation by visual inspection of the generated results reveals that the models, particularly those trained on 32×32 images in Figure 2, can generate combinations of attributes that they did not see during training. There is no difference in perceived quality between images generated using cosine or scaled linear scheduling. Regarding models trained on 64×64 images, the generated images

still contain OOD attribute combinations. Nevertheless, the results confirm the research question, demonstrating that small diffusion models, when implemented and trained from scratch, can generalize combinations of attributes that were not encountered during training. Spatial cross-attention layers successfully direct these independent textual concepts to control local pixel synthesis without memorizing fixed attribute co-occurrences.

3) **Diversity and Seed Sensitivity:** The images in Figure 3 clearly show that the conditioned models are stable with respect to random variations during sampling. However, they are not particularly good at generating novel samples from the same prompt across different seeds, even with the Classifier-Free Guidance and the chosen scale value. The low diversity scores reported in Table III confirm overfitting as a symptom during training.
   For unconditioned models, each inference produces a new image that differs from all previous images. For this reason, the Diversity Scores are extremely out of scale with respect to conditioned models.

B. Challenges Faced and Architectural Trade-offs

Although the experiments address the research questions, the proposed models have limitations in their architecture:

1) **Capacity Balancing of Text Encoder and Denoiser:** One of the primary challenges was tuning the capacity of the models during development. Even before training the models, it was difficult to determine if the proposed structure, the number of Residual and Attention blocks for each U-Net layer, the base channels, and the spatial resolutions were too low or too high.
   Figure 3 shows that, for small images (32×32) neither higher base channels (*compositional-linear-32*) nor a different noise schedule type enhances the model's performance or the quality of the sampled images. This means that a small, contained model easily fulfills the research objective. However, only in *compositional-cosine-32*'s generations there are small variations across seeds, confirming that cosine scheduling enhances generation of small details in final sampling time steps.
   On the other hand, for larger images (64×64), the lower base channels in *compositional-linear-64* demonstrate high capability in reconstructing facial features but introduce a uniform background and colour distortion in avatars that is random and changes across different seeds. Furthermore, cosine scheduling in *compositional-cosine-64* still produces better visual results than scaled linear scheduling in *conditioned-ordinary-64*.
2) **Parameters Counts and Model Sizes:** The results in Table III show that the architecture dimensions are the same for models designed to work with images that are either 32×32 or 64×64. In fact, convolutional layers and attention mechanisms have learnable parameters that are not dependent on input or output image sizes, but are connected to hyperparameters such as base channels, time, and text embedding dimensions.
   Moreover, in this implementation, unconditioned models share the same architecture as conditioned ones. However, during training, a dropout rate of 1 is applied to the input captions. This ensures that the text embeddings remain constant throughout the training process. Consequently, the text encoder participates in the pipeline without learning anything. For this reason, unconditioned models have the same parameter counts and sizes, even though the effective model is smaller.
3) **Sampling Time and VRAM Usage:** By inspecting the graphs in Figure 4 and Figure 5 and comparing them with the values reported in Table IV it is possible to confirm that sampling time depends directly on the size of the output image and the number of base channels. These two hyperparameters directly influence the number of operations required to generate new images. For this reason, *compositional-linear-64* requires less time than other 64×64 models because it has half the number of base channels. The same reasoning applies to *compositional-linear-32*, which has twice the number of base channels of other 32×32 models. Furthermore, the larger the output images and the higher the number of base channels, the more VRAM is required during sampling.
   Finally, increasing the batch size reduces the sampling time per image, as expected, but at the cost of using more VRAM. One exception is unconditioned models, which were expected to be faster and require less VRAM than conditioned models because they only have to process one noisy sample at a time instead of two. However, this does not result in a significant reduction in sampling time or VRAM usage.

C. Recommendations for Improvement and Future Work

Based on our empirical observations and diagnostic checks, we propose the following concrete technical directions to extend this work:

1) **Tuning Guidance Scale for Enhanced Diversity:** As indicated by the low Diversity Scores, high Classifier-Free Guidance (CFG) scale values ($w = 3.5$) tightly constrain the generative manifold around prompt conditions. Future experiments should explore lower CFG scales ($w \in [1.2, 2.0]$) to strike a better balance between prompt fidelity and structural diversity across different random seeds [12].
2) **Early Stopping and Dataset Scaling:** To mitigate the observed overfitting on training captions and pixel features, training should be monitored via early stopping strategies based on validation loss. Additionally, scaling up the training dataset from the 10k subset to larger datasets (e.g., the full 100k Google Cartoon Set [4]) would provide a richer variety of multi-attribute combinations and prevent memorization.
3) **Exponential Moving Average (EMA) for Artifact Elimination:** To eliminate colour distortion, hue shifts, and uniform background artifacts, particularly at 64×64

resolutions, maintaining an exponential moving average (EMA) of model weights during training with a decay factor of 0.9999 is strongly recommended. EMA weights and other implementations, such as EMAG [20] smooth out gradient variance. This stabilizes the generation of background pixels during reverse sampling.

VIII. Conclusion and Future Work

In this paper, we presented a complete, self-contained, pixel-space denoising diffusion probabilistic model (DDPM) that is trained entirely from scratch for text-conditioned cartoon avatar synthesis. It is completely independent of external pretrained backbones or black-box abstractions. Through the design of a custom Text Encoder, a compact cross-attention U-Net denoiser, and a dedicated compositional out-of-distribution (OOD) benchmark protocol, we systematically examined the recombination capabilities of lightweight diffusion models within strict computational constraints.

Both our quantitative results, which were evaluated using FID, KID, and seed diversity metrics, and our qualitative analyses demonstrate that lightweight architectures operating in raw pixel space can effectively factorize multimodal representations and synthesize unseen combinations of visual attributes. Furthermore, our hardware efficiency profiling shows that structured cross-attention conditioning provides robust generative capabilities, even on consumer-grade GPU hardware. Future work will explore deterministic fast samplers (e.g., DDIM) and Flow Matching formulations to accelerate inference. Additionally, we will explore Exponential Moving Average (EMA) weight tracking and dataset scaling to further stabilize visual fidelity.

References

[1] V. Cassone, "Pixel Avatar DDPM: A Tiny Text-Conditioned Avatar Diffusion from Scratch," https://github.com/ValeMX/pixel-avatar-ddpm, 2026, accessed: Oct. 2026.

[2] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *CoRR*, vol. abs/2006.11239, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2006.11239

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," *CoRR*, vol. abs/2112.10752, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2112.10752

[4] Google LLC, "Cartoon set: An image dataset of random cartoons," 2018, accessed: Oct. 14, 2026. [Online]. Available: https://google.github.io/cartoonset/

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1706.03762

[6] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," *CoRR*, vol. abs/2103.00020, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2103.00020

[7] J. Ni, G. H. Ábrego, N. Constant, J. Ma, K. B. Hall, D. Cer, and Y. Yang, "Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models," *CoRR*, vol. abs/2108.08877, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2108.08877

[8] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS 32)*, 2019, pp. 8024–8035. [Online]. Available: https://pytorch.org

[9] Y. Wu and K. He, "Group normalization," *CoRR*, vol. abs/1803.08494, 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1803.08494

[10] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," *CoRR*, vol. abs/2102.09672, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2102.09672

[11] V. Cassone, "Pixel Avatar DDPM: A Tiny Text-Conditioned Avatar Diffusion from Scratch. OneDrive Repository with Checkpoints," https://politecnicobari-my.sharepoint.com/:f:/g/personal/v_cassone1_studenti_poliba_it/IgC_0uJZv-7TS4XsCKA7cdzOAVFJgAFqkywNz83U5gVk95w?e=w8zh6K, 2026, accessed: Oct. 2026.

[12] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *CoRR*, vol. abs/2207.12598, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2207.12598

[13] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, G. Klambauer, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a nash equilibrium," *CoRR*, vol. abs/1706.08500, 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1706.08500

[14] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," *CoRR*, vol. abs/1512.00567, 2015. [Online]. Available: https://doi.org/10.48550/arXiv.1512.00567

[15] L. V. Kantorovich, "Mathematical methods of organizing and planning production," *Management Science*, vol. 6, no. 4, pp. 366–422, 1960. [Online]. Available: https://doi.org/10.1287/mnsc.6.4.366

[16] M. J. Chong and D. A. Forsyth, "Effectively unbiased FID and inception score and where to find them," *CoRR*, vol. abs/1911.07023, 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1911.07023

[17] M. Binkowski, D. J. Sutherland, M. Arbel, and A. Gretton, "Demystifying MMD gans," *CoRR*, vol. abs/1801.01401, 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1801.01401

[18] N. S. Detlefsen, J. Borovec, J. Schock, A. Harsh, T. Koker, L. Di Liello, D. Stancl, C. Quan, M. Grechkin, and W. Falcon, "Torchmetrics - measuring reproducibility in pytorch," *Journal of Open Source Software*, 2 2022. [Online]. Available: https://www.pytorchlightning.ai

[19] S. Zhao, J. Cui, Y. Sheng, Y. Dong, X. Liang, E. Chang, and Y. Xu, "Large scale image completion via co-modulated generative adversarial networks," 03 2021.

[20] A. Yadav, T. D. Huy, and L. Liu, "Emag: Self-rectifying diffusion sampling with exponential moving average guidance," 2026. [Online]. Available: https://arxiv.org/abs/2512.17303

TABLE V
GOOGLE CARTOON SET DATASET ATTRIBUTE MAPPING OVERVIEW

Category	Attribute Name	Description	Variants	Value Mapping Examples
Artwork	chin_length	Length of chin below mouth	3	0: short, 1: medium length, 2: long
	eye_angle	Tilt of the eye inwards or outwards	3	0: tilted inward, 1: neutral, 2: tilted outward
	eye_lashes	Visibility of eyelashes	2	0: no, 1: visible
	eye_lid	Appearance of the eyelids	2	0: normal, 1: single
	eyebrow_shape	Shape of eyebrows	14	0: flat, 1: slightly curved, ..., 13: textured flat
	eyebrow_weight	Line weight of eyebrows	2	0: thin line, 1: thick line
	face_shape	Overall shape of the face	7	0: inverted triangle, 1: oblong, 2: round, 3: oval, 4: heart, 5: diamond, 6: square
	facial_hair	Facial hair type	15	0: stubble, ..., 13: goatee patch, 14: no
	glasses	Glasses type	12	0: aviator, ..., 10: heart-shaped, 11: no
	hair	Head hair style	111	0: bald, 1: shaved, ..., 110: receding hairline
Colours	eye_color	Colour of irises	5	0: brown, 1: blue, 2: green, 3: gray, 4: black
	face_color	Skin colour of the face	11	0: dark brown, ..., 10: pale beige
	glasses_color	Colour of glasses frame	7	0: black, 1: cream, 2: blue, 3: red, 4: gray, 5: white, 6: pink
	hair_color	Colour of hair, facial hair, eyebrows	10	0: blonde, 1: yellow, 2: orange, 3: red, 4: light brown, 5: brown, 6: dark brown, 7: black, 8: gray, 9: white
Proportions	eye_eyebrow_distance	Distance between eye and eyebrow	3	0: medium, 1: far, 2: close
	eye_slant	Eye rotation angle	3	0: slanted up, 1: straight, 2: slanted down
	eyebrow_thickness	Vertical scale of eyebrows	4	0: thin, 1: medium thin, 2: medium thick, 3: thick
	eyebrow_width	Horizontal scale of eyebrows	3	0: narrow, 1: medium width, 2: wide